

PROMPT MESTRE DEFINITIVO

PLATAFORMA SAAS MULTI-TENANT DE AGENDAMENTOS

DESENVOLVIMENTO EM 3 FASES — STACK 100% GRATUITA NO INÍCIO

REGRA ABSOLUTA Nº 1 — CUSTO ZERO NO INÍCIO

Esta é uma regra FUNDAMENTAL deste projeto.

O sistema deve ser desenvolvido inicialmente utilizando somente ferramentas, serviços, APIs, bancos de dados, hospedagem e recursos que possuam uma camada gratuita oficial e utilizável, sem necessidade de contratar planos pagos para colocar o MVP em funcionamento.

O objetivo é desenvolver e colocar o sistema no ar com custo inicial próximo de R\$0.

NÃO utilizar inicialmente:

- * serviços pagos obrigatórios;
- * APIs que exigem assinatura paga;
- * bancos pagos;
- * hospedagem paga;
- * servidores VPS pagos;
- * serviços com cobrança obrigatória;
- * APIs de terceiros que não possuam camada gratuita adequada;
- * ferramentas que dependam de créditos pagos para funcionar;
- * serviços piratas;
- * serviços ilegais;
- * chaves compartilhadas;
- * contas de terceiros;
- * qualquer método para burlar limites de serviços.

IMPORTANTE

“Grátis” significa:

Free Tier oficial fornecido pelo próprio serviço.

Não utilizar métodos para contornar limites ou cobranças.

REGRA ABSOLUTA Nº 2 — FIREBASE SERÁ O BANCO INICIAL

O banco de dados inicial da plataforma deverá ser:

Firebase

Utilizar preferencialmente:

- * Firebase Authentication;
- * Cloud Firestore;
- * Firebase Storage;

- * Firebase Cloud Functions somente quando o free tier permitir;
- * Firebase Hosting se adequado ao projeto;
- * Firebase App Check quando aplicável.

O banco principal será:

Cloud Firestore

NÃO utilizar PostgreSQL ou Supabase como banco principal inicialmente.

A arquitetura deve, entretanto, ser criada de forma suficientemente organizada para permitir uma futura migração caso o projeto cresça.

OBJETIVO

Construir uma única plataforma SaaS de agendamentos.

Eu sou o proprietário da plataforma.

Empresas poderão contratar o sistema e criar seu próprio ambiente.

Cada empresa terá:

- * painel;
- * usuários;
- * clientes;
- * serviços;
- * categorias;
- * profissionais;
- * horários;
- * agenda;
- * agendamentos;
- * pagamentos;
- * notificações;
- * site público;
- * personalização;
- * relatórios;
- * configurações.

Tudo deve fazer parte de:

UM ÚNICO SISTEMA

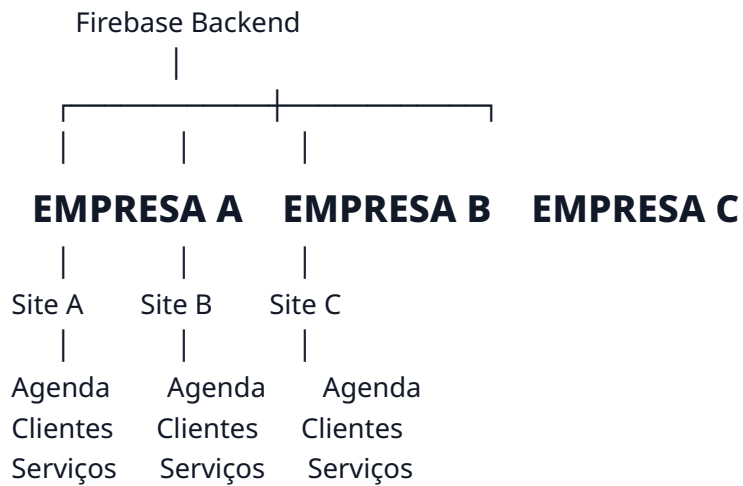
Não criar três projetos.

As três fases são etapas do mesmo produto.

VISÃO

MINHA PLATAFORMA

|



STACK INICIAL OBRIGATÓRIA

Priorizar:

Frontend

- * Next.js;
- * React;
- * TypeScript;
- * Tailwind CSS;

* PWA.

Backend

Utilizar:

- * Next.js;
- * Firebase;
- * Firebase Admin SDK quando necessário;
- * Cloud Functions apenas quando fizer sentido e estiver dentro do free tier.

Banco

Firebase Cloud Firestore

Autenticação

Firebase Authentication

Arquivos

Firebase Storage

Hosting

Preferir:

Vercel Free

ou

Firebase Hosting Free

Escolher uma das opções de acordo com a arquitetura final.

Não contratar servidor pago inicialmente.

REGRA DE DEPENDÊNCIAS

Antes de adicionar qualquer serviço externo, verificar:

1. Existe free tier oficial?
2. O free tier é suficiente para o MVP?
3. É necessário cartão?
4. Existe limite que inviabilize o sistema?
5. Existe alternativa gratuita?
6. É possível implementar a funcionalidade sem serviço externo?

Se existir alternativa gratuita adequada:

usar a alternativa gratuita.

REGRA DE SERVIÇOS EXTERNOS

Para qualquer serviço externo:

SERVIÇO

↓

Possui free tier?

↓

SIM

↓

Verificar limite

↓

Adequado para MVP?

↓

SIM

↓

UTILIZAR

Caso contrário:

Não utilizar inicialmente.

Preparar arquitetura para integração futura.

PAGAMENTOS — IMPORTANTE

Existe uma diferença entre:

Pagamento dos clientes da plataforma

e

Pagamento dos agendamentos dos clientes das empresas.

O sistema deve ser preparado para ambos.

Porém, não inventar um gateway de pagamento gratuito se não existir um adequado.

Na versão inicial:

- * implementar arquitetura de pagamentos;
- * criar interfaces;
- * criar modelos de dados;
- * criar estados;
- * criar webhooks;
- * criar abstração de gateway;
- * deixar integração preparada.

Quando houver um gateway com condições gratuitas adequadas ao cenário, integrar.

Se não houver:

deixar o pagamento real desativado até que as credenciais/serviço sejam configurados.

NUNCA simular um pagamento como se fosse real.

WHATSAPP — IMPORTANTE

A integração oficial com WhatsApp pode possuir custos dependendo do uso.

Portanto:

NÃO utilizar soluções piratas.

NÃO utilizar WhatsApp Web automatizado de maneira insegura.

NÃO utilizar APIs não oficiais para fingir que são oficiais.

Inicialmente:

- * criar sistema de notificações interno;
- * criar e-mail quando houver opção gratuita;
- * criar arquitetura de WhatsApp;
- * criar templates;
- * criar eventos;
- * criar camada de integração.

A integração real poderá ser ativada futuramente.

E-MAIL

Utilizar inicialmente somente um provedor com free tier oficial adequado.

Se nenhum for adequado:

- * implementar abstração;
- * permitir SMTP;
- * deixar configuração externa.

Nunca colocar credenciais no código.

SMS

Não utilizar inicialmente se houver custo.

Utilizar:

- * notificações internas;
 - * e-mail;
 - * WhatsApp futuramente.
-

DOMÍNIO

Inicialmente utilizar:

empresa.minhaplataforma.com

ou estrutura equivalente.

Não comprar domínios individuais para cada cliente.

Domínio personalizado:

www.cliente.com.br

será preparado para fase futura.

FIREBASE — ARQUITETURA

Utilizar:

Firebase

|

├— Authentication

|

├— Firestore

|

├— Storage

|

├— Hosting (se escolhido)

|

└— Functions (quando necessário)

FIRESTORE

Estruturar os dados pensando em multi-tenancy.

Exemplo conceitual:

```
tenants/{tenantId}
tenants/{tenantId}/users/{userId}
tenants/{tenantId}/services/{serviceId}
tenants/{tenantId}/categories/{categoryId}
tenants/{tenantId}/professionals/{professionalId}
tenants/{tenantId}/customers/{customerId}
tenants/{tenantId}/appointments/{appointmentId}
tenants/{tenantId}/payments/{paymentId}
tenants/{tenantId}/notifications/{notificationId}
```

Essa estrutura é apenas uma orientação inicial.

Analise cuidadosamente se subcollections, collections globais ou uma combinação são melhores para cada entidade.

REGRA CRÍTICA DO FIRESTORE

Não criar queries que dependam de varrer milhares de documentos.

Projetar índices desde o início.

Considerar:

- * leituras;
- * escritas;
- * exclusões;
- * tamanho dos documentos;
- * listeners realtime;
- * paginação.

O free tier do Firestore possui limites.

Portanto:

A arquitetura deve ser econômica em leituras e escritas.

ECONOMIA DE FIRESTORE

Evitar:

onSnapshot()

em dezenas de telas simultaneamente sem necessidade.

Evitar:

- * consultas repetidas;
- * listeners permanentes desnecessários;
- * carregar todos os clientes;
- * carregar todos os agendamentos;
- * carregar todas as empresas.

Utilizar:

- * paginação;
 - * filtros;
 - * índices;
 - * cache;
 - * queries específicas;
 - * carregamento sob demanda.
-

FIREBASE SECURITY RULES

Essa é uma das partes MAIS IMPORTANTES.

Criar regras Firestore robustas.

O cliente só pode acessar:

tenant

ao qual pertence.

Exemplo conceitual:

request.auth.uid

↓

tenant membership

↓

tenantId

↓

autorização

Não confiar somente no frontend.

RBAC

Criar:

PLATFORM_OWNER

PLATFORM_ADMIN

TENANT_OWNER

TENANT_ADMIN

MANAGER

PROFESSIONAL

CUSTOMER

As permissões devem ser verificadas:

- * frontend;

- * backend;
 - * Firestore Rules;
 - * Cloud Functions quando aplicável.
-

FASE 1

FUNDAÇÃO + SAAS + AGENDAMENTO

OBJETIVO

Criar o MVP completo e funcional sem custos obrigatórios.

FASE 1.1 — CONFIGURAÇÃO FIREBASE

Criar projeto Firebase.

Configurar:

- * Authentication;
- * Firestore;
- * Storage;
- * Security Rules;
- * índices;
- * ambiente de desenvolvimento;
- * ambiente de produção quando possível.

Criar:

.env.example

Nunca colocar secrets reais no GitHub.

FASE 1.2 — AUTENTICAÇÃO

Implementar:

- * cadastro;
- * login;
- * logout;
- * recuperação de senha;
- * alteração de senha;
- * verificação;
- * persistência de sessão.

Login:

- * e-mail;
- * senha.

Google:

- * opcional;
 - * somente se estiver adequado ao free tier.
-

FASE 1.3 — TENANT

Criar:

tenants
tenant_users
users/profiles

Fluxo:

Usuário

↓

Cadastro

↓

Cria empresa

↓

Tenant criado

↓

Usuário associado

↓

TENANT_OWNER

FASE 1.4 — PAINEL MASTER

Criar painel exclusivo.

Dashboard:

- * empresas;
- * usuários;
- * agendamentos;
- * status;
- * atividade.

Gerenciamento:

- * empresas;
 - * usuários;
 - * tenants.
-

FASE 1.5 — CADASTRO EMPRESA

Campos:

- * nome;
- * nome fantasia;

* **CPF/CNPJ;**

- * telefone;
- * WhatsApp;
- * e-mail;
- * endereço;
- * cidade;
- * estado;

* **CEP;**

- * Instagram;
 - * descrição;
 - * logo.
-

FASE 1.6 — SEGMENTOS

Criar:

- * barbearia;
 - * salão;
 - * estética;
 - * clínica;
 - * odontologia;
 - * personal;
 - * tatuagem;
 - * fotografia;
 - * oficina;
 - * pet;
 - * serviços;
 - * outros.
-

FASE 1.7 — CATEGORIAS

CRUD completo.

FASE 1.8 — SERVIÇOS

CRUD:

- * nome;
- * descrição;
- * preço;

- * duração;
 - * categoria;
 - * imagem;
 - * status.
-

FASE 1.9 — PROFISSIONAIS

Criar:

- * nome;
 - * foto;
 - * descrição;
 - * contato;
 - * serviços;
 - * status.
-

FASE 1.10 — DISPONIBILIDADE

Criar:

- * expediente;
 - * intervalo;
 - * folga;
 - * férias;
 - * bloqueios;
 - * feriados;
 - * exceções.
-

FASE 1.11 — AGENDA

Criar:

- * dia;
- * semana;
- * mês.

Filtros:

- * profissional;
 - * serviço;
 - * status.
-

FASE 1.12 — AGENDAMENTO

Fluxo:

Serviço

↓

Profissional

↓

Data

↓

Horário

↓

Cliente

↓

Confirmação

Não permitir:

- * horário ocupado;
 - * conflito;
 - * fora do expediente;
 - * profissional indisponível.
-

FASE 1.13 — PREVENÇÃO DE DOUBLE BOOKING

Essa funcionalidade é obrigatória.

Dois clientes podem tentar reservar simultaneamente.

O backend precisa validar novamente a disponibilidade.

Utilizar:

- * transações Firestore;
 - * lógica de concorrência;
 - * identificadores de slots quando apropriado.
-

FASE 1.14 — CLIENTES

Criar:

- * cadastro;
 - * edição;
 - * busca;
 - * histórico.
-

FASE 1.15 — SITE PÚBLICO

Cada tenant terá:

tenant.minhaplataforma.com

ou equivalente.

Mostrar:

- * logo;
 - * empresa;
 - * serviços;
 - * profissionais;
 - * horários;
 - * localização;
 - * contato;
 - * botão agendar.
-

FASE 1.16 — PERSONALIZAÇÃO

Permitir:

- * logo;
 - * cores;
 - * banner;
 - * descrição;
 - * tema.
-

FASE 1.17 — RESPONSIVIDADE

Desktop:

- * sidebar;
- * dashboard;
- * tabelas;
- * calendário.

Mobile:

- * navegação adaptada;
 - * cards;
 - * agenda adaptada;
 - * formulários adequados.
-

FASE 1.18 — TESTES

Testar:

Cadastro

Login

Empresa

Serviço

Categoria

Profissional

Horário

Cliente
Agendamento
Cancelamento

E principalmente:

Tenant A

↓

tentativa de acessar Tenant B

↓

NEGADO

FASE 1 — CRITÉRIO DE ACEITE

Só considerar concluída quando:

- * Firebase funciona;
 - * autenticação funciona;
 - * multi-tenancy funciona;
 - * Security Rules funcionam;
 - * RBAC funciona;
 - * painel Master funciona;
 - * empresa funciona;
 - * serviços funcionam;
 - * categorias funcionam;
 - * profissionais funcionam;
 - * horários funcionam;
 - * clientes funcionam;
 - * agenda funciona;
 - * site funciona;
 - * agendamento funciona;
 - * double booking é evitado;
 - * mobile funciona.
-

=====

FASE 2

MONETIZAÇÃO + PAGAMENTOS + GESTÃO + AUTOMAÇÃO

=====

OBJETIVO

Transformar o MVP em produto comercial.

Continuar utilizando serviços gratuitos sempre que possível.

FASE 2.1 — PLANOS

Criar:

FREE

BASIC

PRO

PREMIUM

Os nomes podem ser alterados.

FASE 2.2 — LIMITES

Controlar:

- * profissionais;
 - * agendamentos;
 - * armazenamento;
 - * unidades;
 - * recursos.
-

FASE 2.3 — FEATURE FLAGS

Criar:

payments
whatsapp
custom_domain
reports
loyalty
inventory
multi_branch
api

FASE 2.4 — ASSINATURAS

Criar estrutura:

TRIAL
ACTIVE
PAST_DUE
SUSPENDED
CANCELLED

FASE 2.5 — PAGAMENTO DA PLATAFORMA

Preparar:

Plano

↓

Checkout

↓

Gateway

↓

Webhook

↓

Assinatura

Se o gateway escolhido exigir pagamento obrigatório ou não possuir condição adequada para o início: não ativar cobrança real ainda.

Manter arquitetura pronta.

FASE 2.6 — PAGAMENTO DOS AGENDAMENTOS

Preparar:

*** PIX;**

* cartão;

* sinal;

* pagamento no local.

Utilizar gateway somente quando houver solução adequada.

FASE 2.7 — ABSTRAÇÃO DE PAGAMENTO

Criar interface:

PaymentProvider

Com métodos como:

createPayment()

getPayment()
cancelPayment()
refundPayment()

Assim futuramente será possível trocar:

Gateway A

↓

Gateway B

↓

Gateway C

sem reescrever o sistema.

FASE 2.8 — WEBHOOKS

Criar estrutura:

webhook_events

Implementar:

- * validação;
 - * idempotência;
 - * logs.
-

FASE 2.9 — NOTIFICAÇÕES

Criar sistema interno.

Eventos:

- * agendamento;
 - * confirmação;
 - * cancelamento;
 - * pagamento;
 - * lembrete.
-

FASE 2.10 — E-MAIL

Utilizar provedor com free tier oficial.

Caso não seja viável:

- * criar camada de abstração;
 - * deixar configuração futura.
-

FASE 2.11 — WHATSAPP

Preparar integração oficial.

Não utilizar soluções piratas.

Criar:

- * templates;
 - * eventos;
 - * configuração;
 - * provider abstraction.
-

FASE 2.12 — CRM AVANÇADO

Adicionar:

- * tags;
 - * clientes inativos;
 - * frequência;
 - * total gasto;
 - * observações.
-

FASE 2.13 — CUPONS

Implementar:

- * percentual;
 - * valor;
 - * validade;
 - * limite;
 - * valor mínimo.
-

FASE 2.14 — PROMOÇÕES

Implementar:

- * primeira visita;
 - * horários;
 - * combos;
 - * serviços.
-

FASE 2.15 — FIDELIDADE

Implementar:

pontos
recompensas
histórico

FASE 2.16 — AVALIAÇÕES

Após atendimento:

★★★★★

Comentário

FASE 2.17 — RELATÓRIOS

Criar:

- * faturamento;
 - * agendamentos;
 - * cancelamentos;
 - * no-show;
 - * ocupação;
 - * clientes;
 - * serviços;
 - * profissionais.
-

FASE 2.18 — FINANCEIRO

Criar:

Entradas

- * agendamentos;
- * produtos;
- * pacotes.

Saídas

- * despesas;
 - * fornecedores;
 - * funcionários.
-

FASE 2.19 — COMISSÕES

Calcular automaticamente:

Serviço

↓

Preço

↓
Percentual

↓
Comissão profissional

FASE 2.20 — PERSONALIZAÇÃO AVANÇADA

Adicionar:

- * fontes;
- * cores;
- * banners;
- * galerias;
- * depoimentos;

* **FAQ;**

- * redes sociais;
 - * rodapé;
 - * ordem das seções.
-

FASE 2.21 — SEO

Implementar:

- * title;
 - * description;
 - * Open Graph;
 - * sitemap;
 - * robots;
 - * URLs amigáveis;
 - * SEO local.
-

FASE 2.22 — QR CODE

Gerar QR Code para cada empresa.

FASE 2 — CRITÉRIO DE ACEITE

Ao terminar:

- * SaaS comercialmente utilizável;
- * planos;
- * limites;

* CRM;

- * relatórios;
- * financeiro;
- * cupons;
- * fidelidade;
- * avaliações;
- * personalização;
- * arquitetura de pagamentos;
- * notificações;

* SEO.

Tudo deve continuar funcionando sobre a mesma base Firebase.

FASE 3

ESCALA + RECURSOS AVANÇADOS + PRODUÇÃO

OBJETIVO

Preparar a plataforma para crescimento real.

FASE 3.1 — MULTIUNIDADES

Permitir:

Empresa

- └─ Unidade 1
- └─ Unidade 2
- └─ Unidade 3

FASE 3.2 — DOMÍNIO PERSONALIZADO

Preparar:

www.cliente.com.br

Inicialmente, utilizar soluções gratuitas.

Caso o cliente precise comprar um domínio:

o domínio será responsabilidade do cliente.

A plataforma não deve assumir esse custo inicialmente.

FASE 3.3 — PWA

Criar:

- * manifest;
 - * instalação;
 - * ícones;
 - * experiência mobile.
-

FASE 3.4 — APP

Preparar arquitetura para aplicativo futuro.

Não duplicar regras de negócio.

O aplicativo deverá consumir a mesma API/backend.

FASE 3.5 — CALENDÁRIOS

Preparar integração:

- * Google Calendar;
- * Outlook.

Somente utilizar APIs dentro de condições gratuitas adequadas.

FASE 3.6 — BOT WHATSAPP

Criar arquitetura para:

Cliente

↓

WhatsApp

↓

Bot

↓

API

↓

Disponibilidade

↓

Agendamento

Não utilizar métodos não oficiais.

FASE 3.7 — MARKETING AUTOMÁTICO

Automatizações:

Cliente inativo

↓

Campanha

Aniversário

↓

Cupom

Agendamento amanhã

↓

Lembrete

Atendimento concluído

↓

Avaliação

FASE 3.8 — PACOTES

Criar:

- * pacote;
 - * serviços;
 - * quantidade;
 - * validade;
 - * consumo.
-

FASE 3.9 — ASSINATURAS DOS CLIENTES

Permitir que empresas vendam seus próprios planos.

Exemplo:

Plano mensal

4 atendimentos

R\$99

FASE 3.10 — ESTOQUE

Criar:

- * produtos;
- * fornecedores;
- * entradas;
- * saídas;
- * estoque mínimo;
- * alertas.

FASE 3.11 — VENDAS

Integrar:

Venda

↓

Estoque

↓

Financeiro

FASE 3.12 — API PÚBLICA

Criar:

GET /appointments

POST /appointments

GET /customers

POST /customers

GET /services

GET /professionals

FASE 3.13 — API KEYS

Criar:

- * criação;

- * revogação;

- * permissões;

- * rate limit;

- * logs.

FASE 3.14 — WEBHOOKS PÚBLICOS

Eventos:

appointment.created

appointment.confirmed

appointment.cancelled

payment.approved

customer.created

FASE 3.15 — SUPORTE

Criar:

- * tickets;
 - * mensagens;
 - * prioridade;
 - * status;
 - * histórico.
-

FASE 3.16 — IMPERSONATION

Permitir acesso temporário de suporte.

Sempre registrar:

- * administrador;
 - * tenant;
 - * motivo;
 - * horário;
 - * ações.
-

FASE 3.17 — AUDITORIA

Registrar:

- * login;
 - * alterações;
 - * exclusões;
 - * pagamentos;
 - * permissões;
 - * configurações;
 - * acesso administrativo.
-

FASE 3.18 — SEGURANÇA

Implementar:

- * Firebase Security Rules;

* **RBAC;**

- * rate limiting;
- * validação;
- * sanitização;
- * headers;
- * cookies seguros;
- * proteção de endpoints;

* 2FA quando possível.

FASE 3.19 — LGPD

Implementar:

- * política;
 - * termos;
 - * consentimento;
 - * exportação;
 - * exclusão;
 - * anonimização;
 - * retenção.
-

FASE 3.20 — OTIMIZAÇÃO FIREBASE

Monitorar:

- * leituras;
- * escritas;
- * armazenamento;
- * largura de banda;
- * Functions;
- * listeners.

Reduzir custos de operação.

FASE 3.21 — CACHE

Utilizar cache quando possível.

Objetivo:

menos leitura Firestore

↓

menor consumo

↓

maior performance

FASE 3.22 — PAGINAÇÃO

Todas as listas potencialmente grandes devem possuir paginação.

Exemplos:

- * clientes;
- * agendamentos;

- * empresas;
 - * logs;
 - * notificações.
-

FASE 3.23 — FILAS/JOBS

Para tarefas pesadas:

- * notificações;
- * e-mails;
- * relatórios;
- * webhooks;
- * automações.

Utilizar serviços gratuitos enquanto forem suficientes.

Se não houver solução gratuita adequada:

criar arquitetura preparada para futura implementação.

FASE 3.24 — BACKUP

Utilizar mecanismos gratuitos disponíveis.

Criar estratégia de:

- * backup;
- * exportação;
- * restauração.

Se o backup automatizado avançado exigir serviço pago:

documentar essa limitação e preparar arquitetura para ativação futura.

FASE 3.25 — MONITORAMENTO

Implementar inicialmente ferramentas gratuitas.

Monitorar:

- * erros;
 - * disponibilidade;
 - * APIs;
 - * Firebase;
 - * frontend.
-

FASE 3.26 — CI/CD

Utilizar:

- * GitHub;
- * GitHub Actions quando adequado;
- * Vercel/Firebase.

Pipeline:

Git Push

↓

Lint

↓

Typecheck

↓

Testes

↓

Build

↓

Deploy

FASE 3.27 — TESTES E2E

Testar:

Cadastro

↓

Empresa

↓

Serviço

↓

Profissional

↓

Horário

↓

Site

↓

Cliente

↓

Agendamento

↓

Pagamento

↓

Notificação

↓

Atendimento

↓

Avaliação

FASE 3.28 — TESTE MULTI-TENANT

Obrigatório.

Testar:

Tenant A

↓

tenta acessar

↓

Tenant B

↓

NEGADO

Testar:

- * Firestore;

* API;

- * URLs;

- * parâmetros;

- * IDs;

- * sessões;

- * permissões.

FASE 3.29 — PERFORMANCE

O sistema deve ser preparado para crescer.

Otimizar:

- * Firestore;

- * queries;

- * componentes;

- * imagens;

- * bundle;

- * cache;

- * carregamento.

FASE 3.30 — DOCUMENTAÇÃO

Criar:

README.md

ARCHITECTURE.md

DATABASE.md

FIREBASE.md

SECURITY.md

DEPLOYMENT.md

API.md

TESTING.md

ENVIRONMENT.md

FREE_TIER.md

DOCUMENTO ESPECIAL — FREE_TIER.md

Criar documentação contendo:

Firebase

- * serviços utilizados;
- * limites relevantes;
- * consumo estimado;
- * estratégias de economia.

Hosting

- * limite;
- * consumo.

APIs

Para cada API:

Nome

Finalidade

Free tier

Limite

Necessita cartão?

Necessita conta?

REGRA DE MONITORAMENTO DE CUSTO

Criar no painel Master uma área:

Uso da Plataforma

Mostrar, quando possível:

- * empresas;
- * usuários;
- * agendamentos;
- * armazenamento;
- * uso estimado;
- * limites conhecidos.

Se não for possível obter uma métrica automaticamente:

informar que a métrica não está disponível via API gratuita.

Nunca inventar números.

ALERTAS DE LIMITE

Quando possível:

- Normal
- Próximo do limite
- Limite atingido

Criar alertas antes que a aplicação seja interrompida.

REGRA DE MIGRAÇÃO FUTURA

A plataforma deve ser construída pensando em uma futura evolução.

Quando o projeto crescer, poderá migrar:

Firebase

↓

infraestrutura mais robusta

ou utilizar:

Firebase

+

serviços pagos

Não é necessário implementar essa migração agora.

Mas:

- * separar regras de negócio;
 - * evitar acoplamento excessivo;
 - * criar repositories/services;
 - * abstrair providers externos.
-

ABSTRAÇÕES IMPORTANTES

Criar interfaces para:

AuthProvider

PaymentProvider

EmailProvider

WhatsAppProvider

StorageProvider

NotificationProvider

Assim futuramente será possível trocar o provedor.

Exemplo:

Firebase Auth

↓

Outro Auth Provider

Firebase Storage

↓

Outro Storage

Gateway A

↓

Gateway B

sem reconstruir a aplicação inteira.

REGRA SOBRE FIREBASE

Firebase é a infraestrutura inicial oficial.

Não substituir por Supabase, PostgreSQL ou outro banco durante as três fases, salvo se houver uma limitação técnica crítica que impeça o funcionamento.

Se surgir uma limitação:

1. identificar;
 2. documentar;
 3. verificar alternativa gratuita;
 4. só então propor mudança.
-

REGRA SOBRE SERVIÇOS PAGOS

Durante as três fases:

NÃO contratar serviços pagos automaticamente.

Se uma funcionalidade depender de serviço pago:

FUNCIONALIDADE

↓

Serviço pago necessário?

↓

SIM

↓

Criar arquitetura

↓

Implementar modo desativado/configurável

↓

Documentar

↓

Continuar desenvolvimento

Nunca simular que a funcionalidade está funcionando.

REGRA SOBRE CARTÃO DE CRÉDITO

Priorizar serviços que não exijam cartão para começar.

Se um serviço oferecer free tier, mas exigir cartão obrigatoriamente:

- * informar;
- * procurar alternativa gratuita;

* utilizar alternativa quando possível.

REGRA SOBRE TRIAL

Não considerar:

"14 dias grátis"

como free tier.

Trial não é uma solução gratuita permanente.

Preferir:

free tier oficial.

REGRA SOBRE CÓDIGO

Não gerar:

- * código falso;
- * APIs fake;
- * pagamentos falsos;
- * banco mockado;
- * botões sem função.

Se algo ainda não puder ser conectado:

IMPLEMENTAR INTERFACE

+

IMPLEMENTAR MODELO

+

IMPLEMENTAR FLUXO

+

DEIXAR PROVIDER DESATIVADO

REGRA SOBRE PRODUÇÃO

Mesmo utilizando serviços gratuitos, o sistema precisa ser desenvolvido seguindo padrões profissionais.

Implementar:

- * segurança;
- * logs;
- * tratamento de erros;
- * validação;
- * testes;

- * isolamento;
- * backups;
- * documentação.

“Grátis” não significa “mal feito”.

CHECKLIST FINAL

Firebase

- * ☐ Authentication
- * ☐ Firestore
- * ☐ Storage
- * ☐ Security Rules
- * ☐ Indexes
- * ☐ Functions quando necessário
- * ☐ App Check quando aplicável

SaaS

- * ☐ Multi-tenant

* ☐ **RBAC**

- * ☐ Planos
- * ☐ Limites
- * ☐ Assinaturas
- * ☐ Painel Master

Agendamento

- * ☐ Serviços
- * ☐ Categorias
- * ☐ Profissionais
- * ☐ Horários
- * ☐ Agenda
- * ☐ Clientes
- * ☐ Double booking prevention
- * ☐ Cancelamento
- * ☐ Remarcação

Site

- * ☐ Site público
- * ☐ Personalização

* ☐ **SEO**

- * ☐ Responsivo

* ☐ **PWA**

- * ☐ QR Code

Gestão

* [] CRM

- * [] Relatórios
- * [] Financeiro
- * [] Comissões
- * [] Cupons
- * [] Fidelidade
- * [] Avaliações

Integrações

- * [] Payment abstraction
- * [] Email abstraction
- * [] WhatsApp abstraction
- * [] Calendar abstraction

* [] API

- * [] Webhooks

Segurança

- * [] Firebase Rules

* [] RBAC

- * [] Tenant isolation
- * [] Validação
- * [] Sanitização
- * [] Rate limiting
- * [] Auditoria

* [] LGPD

Infraestrutura

- * [] GitHub

* [] CI/CD

- * [] Deploy gratuito
- * [] Monitoramento
- * [] Backup
- * [] Documentação

ORDEM OBRIGATÓRIA DE EXECUÇÃO

A IA deve seguir:

ANÁLISE

↓

ARQUITETURA

↓

FIREBASE



AUTENTICAÇÃO



MULTI-TENANT



RBAC



PAINEL MASTER



TENANT



SERVIÇOS



PROFISSIONAIS



HORÁRIOS



CLIENTES



AGENDA



AGENDAMENTO



SITE PÚBLICO



PERSONALIZAÇÃO



FASE 1 TESTES



FASE 2



FASE 2 TESTES



FASE 3

↓

TESTES FINAIS

↓

SEGURANÇA

↓

PERFORMANCE

↓

DEPLOY

PRIMEIRO PASSO

Antes de escrever qualquer código:

Analise todo este documento.

Depois apresente:

1. arquitetura;
2. stack;
3. estrutura de pastas;
4. arquitetura Firebase;
5. modelo Firestore;
6. Security Rules;

7. RBAC;

8. estratégia multi-tenant;
9. fluxo de autenticação;
10. fluxo de agendamento;
11. estratégia de economia do Firestore;
12. serviços gratuitos que serão utilizados;
13. limites relevantes;
14. quais funcionalidades precisam de serviços externos;
15. quais funcionalidades ficarão preparadas para futura integração;
16. roadmap das três fases;
17. critérios de aceite;
18. riscos técnicos;
19. plano de deploy gratuito.

Não comece a Fase 2 antes de concluir e testar a Fase 1.

Não comece a Fase 3 antes de concluir e testar a Fase 2.

O resultado final deve ser:

UM ÚNICO SaaS MULTI-TENANT DE AGENDAMENTOS

com:

- * Firebase como infraestrutura inicial;
- * custo inicial mínimo/próximo de zero;
- * ferramentas gratuitas oficiais;
- * arquitetura profissional;
- * segurança;
- * escalabilidade;
- * possibilidade de trocar provedores futuramente;
- * três fases integradas;
- * código real;
- * banco real;
- * autenticação real;
- * agendamento real;
- * preparado para pagamentos e integrações;
- * preparado para evolução futura.

NÃO utilize serviços pagos enquanto existir alternativa gratuita adequada.

NÃO simule funcionalidades reais.

NÃO use métodos para burlar limitações de serviços.

NÃO transforme as três fases em projetos separados.

CONSTRUA UMA ÚNICA PLATAFORMA QUE EVOLUI AO LONGO DAS TRÊS FASES.